



Lab 11

Rivest-Shamir-Adleman (RSA) Encryption

Name: Syed Saad Imran/Huzefa Soni	Student ID: 09370/08940
-----------------------------------	-------------------------

11.1 Objective

The Objectives of this lab are:

- Generating an RSA encryption key pair
- Implementing encryption and decryption using RSA

11.2 Background/Scenario

RSA encryption is a crucial part of digital security. It was created in the late 1970s by Ron Rivest, Adi Shamir, and Leonard Adleman. RSA uses two exponents, e (public) and d (private), along with a large modulus number, n , created during key generation.

Both encryption and decryption rely on modular exponentiation, a process made efficient by algorithms like fast exponentiation. It is crucial to emphasize that while performing modular exponentiation is an efficient process, its inverse operation, involving modular logarithm, is an exceptionally challenging task. This complexity arises because solving modular logarithms is as difficult as factoring the modulus, a problem for which no known polynomial-time algorithm exists.

To put it simply, anyone can easily encrypt data quickly when the public exponent (e) is known. Likewise, with the private exponent (d), decryption is a straightforward process. However, for any unauthorized third party, decrypting the information is nearly impossible due to the formidable computational hurdles involved. This inherent security feature of RSA ensures that sensitive data remains protected from prying eyes.

11.3 Procedure

Figure 1 shows the general idea behind the procedure used in RSA.

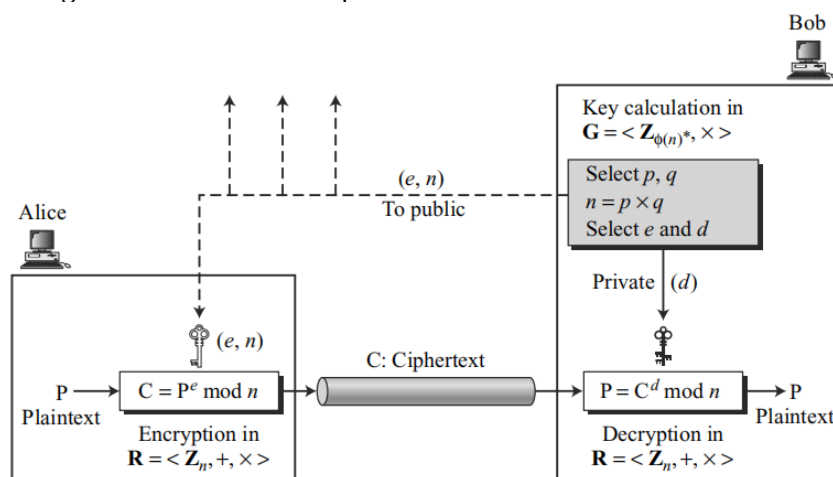


Figure 1: Encryption, Decryption and Key Generation



RSA uses two algebraic structures; a ring and a group.

Encryption/Decryption Ring

Encryption and decryption are done using the commutative ring $R = \langle \mathbb{Z}_n, +, \times \rangle$ with two arithmetic operations: addition and multiplication. In RSA, this ring is public because the modulus n is public. Anyone can send a message to Bob using this ring to do encryption.

Key-Generation Group

RSA uses a multiplicative group $G = \langle \mathbb{Z}_{\phi(n)}^*, \times \rangle$ for key generation. This group supports only multiplication and division (using multiplicative inverses), which are needed for generating public and private keys. This group is hidden from the public because its modulus, $\phi(n)$ is hidden from the public.

Task 1: Generate an RSA encryption key pair

To generate both a public and a private key, you can follow the procedure outlined in Figure 2. Once the keys are generated, the public key is represented as the tuple (e, n) , and the private key is simply the integer d . It is important to note that the values of p , q , and $\phi(n)$ can be safely discarded after key generation, as they will not be required unless there is a need to change the private key without altering the modulus – a practice usually discouraged.

For security, it is recommended that each prime, p or q , should have a size of 512 bits, equivalent to 154 decimal digits. This choice results in a modulus, n , that is 1024 bits in size, or 309 digits long.

```
RSA_Key_Generation
{
    Select two large primes  $p$  and  $q$  such that  $p \neq q$ .
     $n \leftarrow p \times q$ 
     $\phi(n) \leftarrow (p - 1) \times (q - 1)$ 
    Select  $e$  such that  $1 < e < \phi(n)$  and  $e$  is coprime to  $\phi(n)$ 
     $d \leftarrow e^{-1} \bmod \phi(n)$            //  $d$  is inverse of  $e$  modulo  $\phi(n)$ 
    Public_key  $\leftarrow (e, n)$          // To be announced publicly
    Private_key  $\leftarrow d$            // To be kept secret
    return Public_key and Private_key
}
```

Figure 2: RSA key generation algorithm

Create a module that implements the functionality shown in Figure 2.



```
module rsa_keygen #(parameter WIDTH = 1024) (  
    input wire clk,  
    input wire reset,  
    input wire start,  
    input wire [WIDTH/2-1:0] p,  
    input wire [WIDTH/2-1:0] q,  
    input wire [WIDTH-1:0] e,  
    output reg [WIDTH-1:0] n,  
    output reg [WIDTH-1:0] d,  
    output reg done  
);  
    reg [WIDTH-1:0] phi;  
    reg [WIDTH-1:0] k;  
    localparam IDLE = 2'b00;  
    localparam CALC_N_PHI = 2'b01;  
    localparam FIND_D = 2'b10;  
    localparam DONE = 2'b11;  
    reg [1:0] state;  
    always @(posedge clk or posedge reset) begin  
        if (reset) begin  
            state <= IDLE;  
            done <= 0;  
            n <= 0;  
            d <= 0;  
        end else begin  
            case (state)  
                IDLE: begin  
                    done <= 0;  
                    if (start) begin  
                        state <= CALC_N_PHI;  
                    end  
                end  
                CALC_N_PHI: begin  
                    n <= p * q;  
                    phi <= (p - 1) * (q - 1);  
                    k <= 1;  
                    state <= FIND_D;  
                end  
                FIND_D: begin  
                    if (((k * e) % phi) == 1) begin  
                        d <= k;  
                        state <= DONE;  
                    end  
                end  
            endcase  
        end  
    end
```



```
end else begin
    k <= k + 1;
end
end

DONE: begin
    done <= 1;
    state <= IDLE;
end
endcase
end
end
endmodule
```

Task 2: RSA Encryption

Messages can be sent by anyone using this public key. In RSA, the encryption process can be executed using an algorithm that operates with polynomial time complexity, as demonstrated in Figure 3. It is worth noting that the size of the plaintext must be smaller than n . If the plaintext exceeds this limit, it should be divided into blocks for proper encryption.

```
RSA_Encryption ( $P, e, n$ )           //  $P$  is the plaintext in  $Z_n$  and  $P < n$ 
{
     $C \leftarrow \text{Fast\_Exponentiation}(P, e, n)$     // Calculation of  $(P^e \bmod n)$ 
    return  $C$ 
}
```

Figure 3: RSA encryption

Create a module that implements the functionality shown in Figure 3.



```
`timescale 1ns / 1ps

module rsa_encryption #(parameter WIDTH = 1024) (
    input wire clk,
    input wire reset,
    input wire start,
    input wire [WIDTH-1:0] message,
    input wire [WIDTH-1:0] e,
    input wire [WIDTH-1:0] n,
    output wire [WIDTH-1:0] ciphertext,
    output wire done
);

    // Instantiate the Core Engine for Encryption
    square_and_multiply #(WIDTH) encrypt_engine (
        .clk(clk),
        .reset(reset),
        .start(start),
        .base(message),
        .exp(e),
        .mod(n),
        .result(ciphertext),
        .done(done)
    );

endmodule
```



```
`timescale 1ns / 1ps

module square_and_multiply #(parameter WIDTH = 1024) (
    input wire clk,
    input wire reset,
    input wire start,
    input wire [WIDTH-1:0] base,
    input wire [WIDTH-1:0] exp,
    input wire [WIDTH-1:0] mod,
    output reg [WIDTH-1:0] result,
    output reg done
);

    reg [WIDTH*2-1:0] base_reg;
    reg [WIDTH-1:0] exp_reg;
    reg [WIDTH*2-1:0] res_reg;

    localparam IDLE = 2'b00;
    localparam CALC = 2'b01;
    localparam DONE = 2'b10;

    reg [1:0] state;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            state <= IDLE;
            result <= 0;
            done <= 0;
        end else begin
            case (state)
                IDLE: begin
                    done <= 0;
                    if (start) begin
                        base_reg <= base % mod;
                        exp_reg <= exp;
                        res_reg <= 1;
                        state <= CALC;
                    end
                end
                CALC: begin
                    if (exp_reg > 0) begin
                        // Multiply step

```



```
        if (exp_reg[0] == 1'b1) begin
            res_reg <= (res_reg * base_reg) % mod;
        end
        // Square step
        base_reg <= (base_reg * base_reg) % mod;
        exp_reg <= exp_reg >> 1;
    end else begin
        state <= DONE;
    end
end
end

DONE: begin
    result <= res_reg[WIDTH-1:0];
    done <= 1;
    state <= IDLE;
end
endcase
end
end
endmodule
```

Task 3: RSA Decryption

The receiver can decrypt the received ciphertext message using the algorithm in figure 4. The RSA decryption process employs an algorithm with polynomial time complexity. It is important to highlight that the size of the ciphertext is always smaller than the modulus, n .

RSA_Decryption (C, d, n)	//C is the ciphertext in Z_n
{	
$P \leftarrow \text{Fast_Exponentiation}(C, d, n)$	// Calculation of $(C^d \bmod n)$
return P	
}	

Figure 4: RSA decryption

Create a module that implements the functionality shown in Figure 4.



```
`timescale 1ns / 1ps

module rsa_decryption #(parameter WIDTH = 1024) (
    input wire clk,
    input wire reset,
    input wire start,
    input wire [WIDTH-1:0] ciphertext,
    input wire [WIDTH-1:0] d,
    input wire [WIDTH-1:0] n,
    output wire [WIDTH-1:0] message,
    output wire done
);

    // Instantiate the Core Engine for Decryption
    square_and_multiply #(WIDTH) decrypt_engine (
        .clk(clk),
        .reset(reset),
        .start(start),
        .base(ciphertext),
        .exp(d),
        .mod(n),
        .result(message),
        .done(done)
    );

endmodule
```



```
`timescale 1ns / 1ps

module square_and_multiply #(parameter WIDTH = 1024) (
    input wire clk,
    input wire reset,
    input wire start,
    input wire [WIDTH-1:0] base,
    input wire [WIDTH-1:0] exp,
    input wire [WIDTH-1:0] mod,
    output reg [WIDTH-1:0] result,
    output reg done
);

    reg [WIDTH*2-1:0] base_reg;
    reg [WIDTH-1:0] exp_reg;
    reg [WIDTH*2-1:0] res_reg;

    localparam IDLE = 2'b00;
    localparam CALC = 2'b01;
    localparam DONE = 2'b10;

    reg [1:0] state;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            state <= IDLE;
            result <= 0;
            done <= 0;
        end else begin
            case (state)
                IDLE: begin
                    done <= 0;
                    if (start) begin
                        base_reg <= base % mod;
                        exp_reg <= exp;
                        res_reg <= 1;
                        state <= CALC;
                    end
                end
                CALC: begin
                    if (exp_reg > 0) begin
                        // Multiply step
                        ...
                    end
                end
            endcase
        end
    end
endmodule
```



```
        if (exp_reg[0] == 1'b1) begin
            res_reg <= (res_reg * base_reg) % mod;
        end
        // Square step
        base_reg <= (base_reg * base_reg) % mod;
        exp_reg <= exp_reg >> 1;
    end else begin
        state <= DONE;
    end
end
end

DONE: begin
    result <= res_reg[WIDTH-1:0];
    done <= 1;
    state <= IDLE;
end
endcase
end
end
endmodule
```

Task 4: Verifying Implementation

Test your implementation by performing the following steps

1. Create an RSA key.
2. Encrypt a message with this key.
3. Decrypt the resulting ciphertext.
4. Compare the decrypted message with the original message.

For submission, please include all your Verilog files and a PDF document containing modules, results, and explanations where necessary



```
module rsa_tb();

    parameter WIDTH = 1024;

    reg clk;
    reg reset;

    // RSA Key Variables
    reg [WIDTH-1:0] p;
    reg [WIDTH-1:0] q;
    reg [WIDTH-1:0] n;
    reg [WIDTH-1:0] e;
    reg [WIDTH-1:0] d;

    // Encrypt signals
    reg enc_start;
    reg [WIDTH-1:0] plaintext_msg;
    wire [WIDTH-1:0] ciphertext;
    wire enc_done;

    // Decrypt signals
    reg dec_start;
    wire [WIDTH-1:0] decrypted_message;
    wire dec_done;

    // Expected ciphertext from the image to verify
    reg [WIDTH-1:0] expected_ciphertext;

    // Clock generation (10ns period)
    always #5 clk = ~clk;

    // Instantiate Task 2: Encryption Wrapper
    rsa_encryption #(WIDTH) u_encrypt (
        .clk(clk), .reset(reset), .start(enc_start),
        .message(plaintext_msg), .e(e), .n(n),
        .ciphertext(ciphertext), .done(enc_done)
    );

    // Instantiate Task 3: Decryption Wrapper
    rsa_decryption #(WIDTH) u_decrypt (
        .clk(clk), .reset(reset), .start(dec_start),
        .ciphertext(ciphertext), .d(d), .n(n),
        .message(decrypted_message), .done(dec_done)
    );
endmodule
```



```
.message(decrypted_message), .done(dec_done)
);

initial begin
    // Initialize
    clk = 0; reset = 1;
    enc_start = 0; dec_start = 0;

    // --- 1. Load the exact values from the images ---

    // Primes (p and q)
    p = 1024'd9613034531358350457419158128061542790930984559499621582258315087964794045505647063849125716018034750312098666606492420191808780667421096063354219926661209;
    q = 1024'd12060191957231446918276794204450896001555925054637033936061798321731482148483764659215389453209175225273226830107120695604602513887145524969000359660045617

    // Public Key (e)
    e = 1024'd35535;

    // Modulus (n) - 309 digits
    n = 1024'd11593504173967614968892509864615887523771457375454144775485526137614788540832635081727687881596832516846884930062548576411125016241455233918292716250765677

    // Private Key (d) - 308 digits
    d = 1024'd58008302860037763936093661289677917594669062089650962180422866111380593852822358731706286910030021710859044338402170729869087600611530620252495988444804756

    // Plaintext (P)
    plaintext_msg = 1024'd1907081826081826002619041819;

    // The Ciphertext (C) shown in the image, used for verification
    expected_ciphertext = 1024'd47530912364622682720636555061054518094237179607049171652323924305445296061319932856661784341835911415119741125200568297979457173603610127

    #20 reset = 0;

    // --- STEP 1: Display Loaded Keys ---
    $display("---- Starting Simulation (Image Test Case) ----");
    $display("Loaded 512-bit Prime p: %0d", p);
    $display("Loaded 512-bit Prime q: %0d", q);
    $display("Calculated Modulus n: %0d", n);

    // --- STEP 2: Encrypt Message ---
    #20;
    $display("---- Starting Task 2: Encryption ----");
    $display("Loaded Plaintext (P): %0d", plaintext_msg);
    enc_start = 1;
    #10 enc_start = 0;
```



```
// --- STEP 2: Encrypt Message ---
#20;
$display("--- Starting Task 2: Encryption ---");
$display("Loaded Plaintext (P): %0d", plaintext_msg);
enc_start = 1;
#10 enc_start = 0;

wait(enc_done);

// Verify Ciphertext matches the image
if (ciphertext == expected_ciphertext) begin
    $display("SUCCESS: Calculated Ciphertext EXACTLY matches the image!");
    $display("Generated Ciphertext: %0d", ciphertext);
end else begin
    $display("ERROR: Calculated Ciphertext does not match the image.");
    $display("Got: %0d", ciphertext);
end

// --- STEP 3: Decrypt Message ---
#20;
$display("--- Starting Task 3: Decryption ---");
dec_start = 1;
#10 dec_start = 0;

wait(dec_done);
$display("Decrypted Message: %0d", decrypted_message);

// --- STEP 4: Verification ---
if (plaintext_msg == decrypted_message) begin
    $display("SUCCESS: True 1024-bit Decryption recovered the original plaintext!");
end else begin
    $display("ERROR: Decrypted message does not match plaintext.");
end

#50 $finish;
end
endmodule
```



Name	Value	0 ps	1 ps	2 ps	3 ps	4 ps	5 ps	6 ps	7 ps
clk	0								
reset	0								
> p[1023:0]	9613034531358350457419	9613034531358350457419158128061542790930984559499621582258315087964794045505647063849125716018034750312098							
> q[1023:0]	1206019195723144691827	1206019195723144691827679420445089600155592505463703393606179832173148214848376465921538945320917522527322							
> n[1023:0]	-1159350417396761496889	-115935041739676149688925098646158875237714573754541447754855261376147885408326350817276878815968325168468							
> e[1023:0]	35535	35535							
> d[1023:0]	5800830286003776393609	5800830286003776393609366128967791759466906208965096218042286611138059385282235873170628691003002171085904							
enc_start	0								
> plaintext_msg[1023:0]	1907081826081826002619	1907081826081826002619041819							
> ciphertext[1023:0]	4753091236462268272063	0							
enc_done	0								
dec_start	0								
> decrypted_message[1023:0]	0	0							
dec_done	0								
> expected_ciphertext[1023:0]	4753091236462268272063	4753091236462268272063655506105451809423717960704917165232392430544529606131993285666178434183591141511974							
> WIDTH[31:0]	00000400	00000400							



Assessment Rubric
Lab 11
Rivest-Shamir-Adleman (RSA) Encryption

Name:	Student ID:
--------------	--------------------

Points Distribution

Task No.	LR 2 Code	LR5 Results	LR9 Report
Task 1	30		
Task 2	20		
Task 3	20		
Task 4		20	
Total	/70	/20	/10
CLO Mapped	CLO 2	CLO 2	CLO2

CLO	Total Points	Points Obtained
2	100	
Total	100	

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.



Lab Evaluation Assessment Rubric

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
LR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.